

Week 7. Problem set (solutions by Karim Khabibrakhmanov)

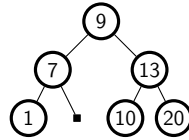
1. Insert the following keys into an initially empty AVL tree [GTG, §11.3]:

2, 3, 29, 5, 11, 23, 13, 17, 19, 7

For each insertion:

- show the *state of the tree* **after** the insertion
- specify the *number of rotations* performed during the insertion

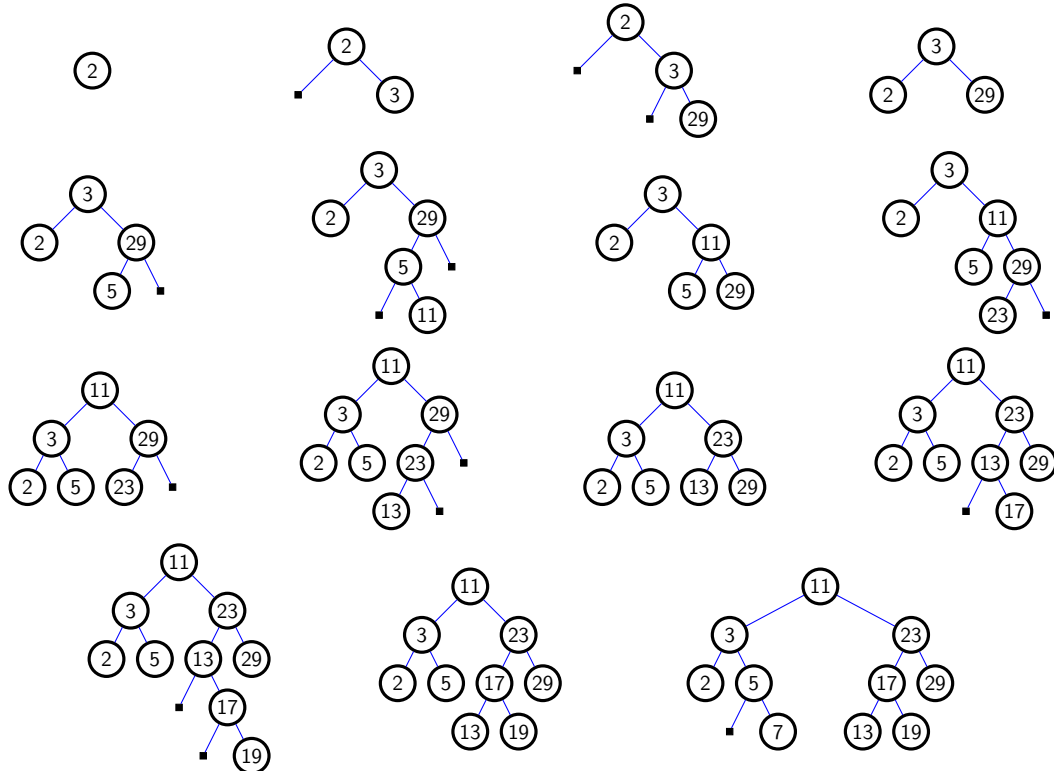
Depict each tree using the array representation for binary trees. For example, consider the following AVL tree:



The tree above must be depicted as the following array:

9	7	13	1	-	10	20
---	---	----	---	---	----	----

Answer:



The tables contains only balanced arrays:

- 1)

2

 – 0 rotation at this step
- 2)

2	-	3
---	---	---

 – 0 rotation at this step
- 3)

3	2	29
---	---	----

 – 1 rotation at this step
- 4)

3	2	29	-	-	5	-
---	---	----	---	---	---	---

 – 0 rotation at this step

- 5)

3	2	11	-	-	5	29
---	---	----	---	---	---	----

 – 2 rotation at this step
- 6)

11	3	29	2	5	23	-
----	---	----	---	---	----	---

 – 1 rotation at this step
- 7)

11	3	23	2	5	13	29
----	---	----	---	---	----	----

 – 1 rotation at this step
- 8)

11	3	23	2	5	13	29	-	-	-	-	-	17	-	-
----	---	----	---	---	----	----	---	---	---	---	---	----	---	---

 – 0 rotation at this step
- 9)

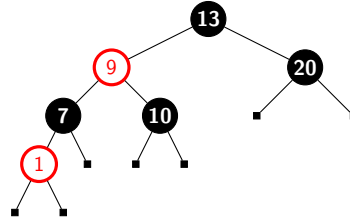
11	3	23	2	5	17	29	-	-	-	-	13	19	-	-
----	---	----	---	---	----	----	---	---	---	---	----	----	---	---

 – 1 rotation at this step
- 10)

11	3	23	2	5	17	29	-	-	-	7	13	19	-	-
----	---	----	---	---	----	----	---	---	---	---	----	----	---	---

 – 0 rotation at this step
- The number of rotations is 6**

Perform the following operations starting with a red-black tree [CLRS, §13] presented below:



- insert 18, 5, 7
- insert 25, 20, 24
- insert 23, 22, 21
- delete 5, 9, 23
- delete 22, 31, 1

For each triple of operations (insertions or deletions):

- show the *state of the tree* **after** the operations
- specify the total *number of rotations* performed during the operations

Depict each tree using the array representation for binary trees. You **must** color the keys for red nodes with **red** and keys for black nodes with **black** color. For example, the initial red-black tree presented above is depicted as the following array:

13	9	20	7	10	-	-	1	-	-	-	-	-	-	-
----	---	----	---	----	---	---	---	---	---	---	---	---	---	---

Answer: Total of 12 rotations

- 1) **Total of 3 rotations and 3 rotations at step 1**

9	5	13	1	7	10	20	-	-	-	7	-	-	18	-
---	---	----	---	---	----	----	---	---	---	---	---	---	----	---

- 2) **Total of 5 rotations and 2 rotations at step 2**

9	5	13	1	7	10	20	-	-	-	7	-	-	18	24
---	---	----	---	---	----	----	---	---	---	---	---	---	----	----

-	-	-	-	-	-	-	-	-	-	-	-	-	-	20	25
---	---	---	---	---	---	---	---	---	---	---	---	---	---	----	----

- 3) **Total of 8 rotations and 3 rotations at step 3**

9	5	20	1	7	13	24	-	-	-	7	10	18	22	25
---	---	----	---	---	----	----	---	---	---	---	----	----	----	----

-	-	-	-	-	-	-	-	-	-	-	-	-	20	23	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	21	-	-	-	-	-	-

- 4) **Total of 12 rotations and 4 rotations at step 4**

20	7	24	7	13	21	25	1	-	10	18	20	22	-	-
----	---	----	---	----	----	----	---	---	----	----	----	----	---	---

- 5) **Total of 12 rotations and 0 rotations at step 5**

20	7	24	7	13	21	25	-	-	10	18	20	-	-	-
----	---	----	---	----	----	----	---	---	----	----	----	---	---	---

Compare randomly-built binary search trees and randomly chosen binary search trees for size $n = 4$:

- (a) Write down the number of distinct shapes for a binary search trees of size $n = 4$.

Answer: 14

- (b) What is the average height of a randomly chosen binary search tree of size $n = 4$?

Answer: $\frac{18}{7} = 2\frac{4}{7} \approx 3$

- (c) Consider $a < b < c < d$. For every permutation of a, b, c, d , write down an array representation of a corresponding binary search tree built from that permutation (by inserting keys in the given order). **Answer:**

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
a	-	b	-	-	-	c	-	-	-	-	-	-	-	d
a	-	b	-	-	-	d	-	-	-	-	-	-	c	-
a	-	c	-	-	b	d								
a	-	c	-	-	b	d								
a	-	d	-	-	b	-	-	-	-	-	-	c	-	-
a	-	d	-	-	c	-	-	-	-	-	b	-	-	-
b	a	c	-	-	-	d								
b	a	d	-	-	c	-								
b	a	c	-	-	-	d								
b	a	c	-	-	-	d								
b	a	d	-	-	c	-								
b	a	d	-	-	c	-								
c	a	d	-	b	-	-								
c	a	d	-	b	-	-								
c	b	d	a	-	-	-								
c	b	d	a	-	-	-								
c	a	d	-	b	-	-								
c	b	d	a	-	-	-								
d	a	-	-	b	-	-	-	-	-	c	-	-	-	-
d	a	-	-	c	-	-	-	-	b	-	-	-	-	-
d	b	-	a	c	-	-								
d	b	-	a	c	-	-								
d	c	-	a	-	-	-	-	b	-	-	-	-	-	-
d	c	-	b	-	-	-	a	-	-	-	-	-	-	-

- (d) What is the average height of a randomly built binary search tree of size $n = 4$?

Answer: $\frac{56}{24} = 2\frac{1}{3} \approx 2$

- (e) Explain, in your own words, why we should not start with a complete binary tree of size n (for any n) with “empty” nodes (i.e. each node does not have any key) and then populate it with given keys $k_1, k_2, \dots, k_n$ to achieve the optimal height of the resulting binary search tree. **Explanation:**

Starting with a complete binary tree of size n and filling it with keys is a bad idea. If the keys are added in the wrong order (e.g., sorted), the tree becomes unbalanced (like a list), making search slow. This method ignores key order, so it's not suitable for an efficient BST.

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.
- [GTG] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser. *Data Structures and Algorithms in Java*. WILEY 2014.